UTILITIES

🚀 Deploying GateOverflow Chatbot & sharing a URL with your team

Source: DEPLOY.md

This app is now a **single-container web service**: the FastAPI backend serves both the API **and** the built React UI on **one port / one URL**, with the prebuilt vector store baked in. Pick a path below.

> ℹ️ **Why you must do the final click, not me:** deploying requires *your* cloud account, billing, and **your OpenAI API key**. I won't place your key on infrastructure I control. Everything below is prepared and copy-paste ready.

What gets deployed

- **Code:** `api/`, `src/`, `scripts/`
- **Built React UI:** compiled inside the image (Dockerfile.prod) and served by FastAPI
- **Retrieval assets:** `storage/chroma` (embeddings) + `storage/pyq_urls.json` (PYQ links) — **baked into the image so no re-embedding runs on the server**
- **Your OpenAI key:** provided at **runtime as a secret env var** (never baked in, never committed, never sent to the browser)

The 137 MB source PDFs in `data/GATA-Data/` are **not** needed at runtime (only to build `storage/`), so they're excluded.

0. One-time prep (on your machine)

Make sure the retrieval assets exist (you've already built these):

```bash
cd 18.Utilities/E.GATE-CS-DA-Chatbot
source .venv/bin/activate
python scripts/ingest.py            # builds storage/chroma  (needs OPENAI_API_KEY)
python scripts/build_pyq_index.py   # builds storage/pyq_urls.json
ls storage/                          # should show chroma/ and pyq_urls.json
```

Test the production image locally first

```bash
docker build -f Dockerfile.prod -t gateoverflow-chatbot .
docker run --rm -p 8000:8000 -e OPENAI_API_KEY="sk-..." gateoverflow-chatbot
# open http://localhost:8000  → the full app (UI + API) on one URL
```

If that works locally, any of the paths below will work.

🔒 Security (read before sharing a public URL)

A shared URL means **anyone who opens it spends your OpenAI credits.** So:

1. **Put the URL behind team-only auth** — easiest is **Cloudflare Access** (email one-time-PIN or Google login, free) in front of the app. Steps below.
2. **Set a hard spending limit** on your OpenAI account (platform.openai.com → Settings → Limits).
3. **Keep the key a secret env var** (the steps below do this).
4. Optionally restrict origins via `CORS_ORIGINS` (not needed for the single-origin container, but available).

★ Recommended (free • long-term • secure): Hugging Face Spaces

Best fit for this app: **truly free** (not trial credits), **2 vCPU / 16 GB RAM** (handles Chroma easily), runs your **Docker** image as-is, automatic **HTTPS**, **private** Spaces + **encrypted secrets** for security. The standard home for AI demos.

Steps

1. **Create the Space:** huggingface.co → *New Space* → SDK **Docker** → Visibility **Private** → name it e.g. `gateoverflow-chatbot`.
2. **Make sure `storage/` is built** locally (Section 0 above): it must contain `chroma/` and `pyq_urls.json` (baked into the image so retrieval works with no server-side embedding).
3. **Push the app into the Space repo:**

```bash
# install git-lfs once (the vector store has files > 10 MB)
git lfs install
git clone https://huggingface.co/spaces/<your-username>/gateoverflow-chatbot
cd gateoverflow-chatbot

APP=../   # path to 18.Utilities/E.GATE-CS-DA-Chatbot
cp -R "$APP"/api "$APP"/src "$APP"/scripts "$APP"/frontend "$APP"/storage .
cp "$APP"/requirements.txt "$APP"/.dockerignore .
cp "$APP"/Dockerfile.prod ./Dockerfile                          # HF builds "Dockerfile"
cp "$APP"/deploy/huggingface-space-README.md ./README.md      # Space metadata

git lfs track "storage/**"
git add -A && git commit -m "Deploy GateOverflow Chatbot" && git push
```

4. **Add your key as a secret:** Space → **Settings** → **Variables and secrets** → *New secret* → `OPENAI_API_KEY = sk-...` (optionally a *Variable* `CHAT_MODEL = gpt-4o-mini`).
5. **Invite your team:** Space → **Settings** → **Sharing** → add teammates (they need free HF accounts). Only invited members can open a private Space.
6. The Space builds automatically. Your URL: `https://<your-username>-gateoverflow-chatbot.hf.space` — share it.

> Notes: free CPU Spaces **pause after ~48 h of inactivity** — just click *Restart* on the Space page (your baked content is intact). Runtime disk is ephemeral, so chat history/bookmarks reset on rebuild (fine for a demo; set `DATABASE_URL` to a hosted Postgres if you want them to persist). Always set an OpenAI **spending limit** as a backstop.

Path A — ⚡ Fastest: Cloudflare Tunnel (your machine hosts it)

Best for a quick team trial. Gives an instant HTTPS URL; your machine stays on.

1. Run the app (the local Docker test above, on `:8000`).
2. Install cloudflared and open a tunnel:

```bash
brew install cloudflared                        # macOS
cloudflared tunnel --url http://localhost:8000
```

   It prints a public URL like `https://<random>.trycloudflare.com` — **share that**. (This quick URL is unguessable but has no auth and changes each run.)

3. **Make it secure & stable (recommended):** with a (free) Cloudflare account + a domain on Cloudflare, create a **named tunnel** and add **Cloudflare Access** so only your teammates' emails can open it:

```bash
cloudflared tunnel login
cloudflared tunnel create gatebot
cloudflared tunnel route dns gatebot chatbot.yourdomain.com
cloudflared tunnel run --url http://localhost:8000 gatebot
```

Then in the Cloudflare dashboard → **Zero Trust** → **Access** → **Applications**, add `chatbot.yourdomain.com` with an email-OTP policy for your team. Share `https://chatbot.yourdomain.com`.

Path B — ☁️ Always-on: Fly.io (recommended managed hosting)

Fly builds from your **local** directory, so the baked-in `storage/` ships automatically. HTTPS URL, scales to zero when idle.

```bash
brew install flyctl
fly auth login
fly launch --no-deploy --copy-config        # uses fly.toml; pick a unique app name & region
fly secrets set OPENAI_API_KEY="sk-..."      # stored encrypted, injected at runtime
fly deploy                                   # builds Dockerfile.prod (incl. storage/)
```

Your URL: `https://<your-app-name>.fly.dev` — share it. Put **Cloudflare Access** (or Fly's networking options) in front for team-only auth.

> Memory: if it OOMs while loading Chroma, bump `memory_mb` to `2048` in `fly.toml` and `fly deploy` again.

Path C — Render / Railway / Google Cloud Run (pre-built image)

These build from your **git repo**, which does **not** include `storage/` (git-ignored). So push a **pre-built image** (with `storage/` baked) to a registry, then deploy that image:

```bash
# Build & push (GitHub Container Registry example)
docker build -f Dockerfile.prod -t ghcr.io/<you>/gateoverflow-chatbot:latest .
docker push ghcr.io/<you>/gateoverflow-chatbot:latest
```

- **Render:** New → Web Service → *Deploy an existing image* → `ghcr.io/<you>/gateoverflow-chatbot:latest`; add env var `OPENAI_API_KEY` (secret); health-check path `/health`. Free/Starter plan gives an HTTPS URL.
- **Railway:** New → Deploy from image → same; add the `OPENAI_API_KEY` variable.

- **Cloud Run:** `gcloud run deploy gateoverflow-chatbot --image ghcr.io/<you>/gateoverflow-chatbot:latest --port 8000 --set-env-vars CHAT_MODEL=gpt-4o-mini --set-secrets OPENAI_API_KEY=openai-key:latest --allow-unauthenticated` (use Cloud Run's IAM / IAP for team-only access).

> ! **Cloudflare Pages/Workers alone can't host this backend** (it's a Python service with a vector store, not static/edge JS). Use Cloudflare for the **frontend CDN / Tunnel / Access**, and Fly/Render/Cloud Run/VPS for the app.

Configuration (env vars)

Var	Purpose
`OPENAI_API_KEY`	**Required**, secret.
`CHAT_MODEL`	e.g. `gpt-4o-mini` (cheap) or `gpt-4o`.
`CORS_ORIGINS`	comma-separated allowed origins; `*` default (single-origin needs none).
`RERANK`	`true` to enable cross-encoder reranking (needs `flashrank`).
`DATABASE_URL`	default SQLite; set Postgres for persistent multi-instance history.
`FRONTEND_DIST`	where the built UI lives (preset in the image).

> The container's SQLite (history/bookmarks/feedback) is **ephemeral** unless you attach a volume or set `DATABASE_URL` to Postgres — fine for a demo; data resets on redeploy.

How your team uses it

1. Open the shared URL (and sign in if you added Cloudflare Access).
2. The **in-app guide** pops up on first visit (reopen via the **?** in the header).
3. They can chat, practice PYQs/mock tests, use flashcards/planner/dashboard, ask about the GateOverflow platform, etc. — everything in the demo videos.

Refreshing content later

Re-build `storage/` locally (`python scripts/ingest.py` after adding material, then `python scripts/build_pyq_index.py`), rebuild the image, and redeploy.